1. What is the role of Istio in a Kubernetes environment, and how does the sidecar proxy model work?

Istio is a service mesh that manages service-to-service communication inside a Kubernetes cluster. Instead of developers implementing networking logic such as retries, timeouts, circuit breaking, encryption, and monitoring inside application code, Istio moves these concerns into the infrastructure layer. It provides:

- Traffic management (routing, load balancing, canary releases)
- Security (mTLS, authentication, authorization)
- Observability (metrics, logs, tracing)
- Policy enforcement This separation allows teams to manage networking consistently without modifying application code.
- **How the Sidecar Proxy Model Works:** Istio uses the sidecar proxy pattern. For every application pod, an Envoy proxy container is injected alongside the main application container.
- **How it works:**

  1. All inbound and outbound traffic from the application flows through the Envoy sidecar.
  2. The sidecar communicates with Istio's control plane (Istiod) to receive configuration.
  3. Traffic rules, security policies, and telemetry are enforced at the proxy level.

  Communication flow: Service A → Envoy Sidecar → Network → Envoy Sidecar → Service B
- **Problems Solved Compared to Application-Level Networking:**

  Without Istio: -
  - Each service must implement retry logic, timeouts, and circuit breaking.
  - TLS must be implemented and maintained per application.
  - Observability must be manually instrumented.
  - Inconsistent implementation across teams.

  With Istio: -
  - Uniform traffic policies across all services.
  - Automatic mutual TLS.
  - Centralized observability.
  - No code changes required.
  - Language-agnostic networking features. It reduces duplication, improves security posture, and ensures consistency.

2. What is the difference between PeerAuthentication and AuthorizationPolicy in Istio, and how would you enforce strict mTLS across all services in a namespace?

PeerAuthentication controls how workloads authenticate each other at the transport layer. It defines whether mutual TLS (mTLS) is:

- STRICT – Only mTLS traffic allowed.
- PERMISSIVE – Accept both mTLS and plaintext.
- DISABLE – No mTLS. This controls how traffic is encrypted and authenticated between services.

AuthorizationPolicy controls who can access what after authentication. It defines:

- Which principals (service accounts)

- can an access which workloads
- Using which HTTP methods, paths, or ports It operates at Layer 7 and enforces access control rules. Enforcing Strict mTLS Across a Namespace

1. Apply a PeerAuthentication resource in that namespace with STRICT mode.
2. Ensure no DestinationRules override TLS settings incorrectly.

Example:

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default
  namespace: your-namespace
spec:
   mtls:
       mode: STRICT
```

This ensures all services in the namespace only accept encrypted traffic using mTLS.

3. How does Istio's traffic management work, and how would you configure a canary deployment using VirtualService and DestinationRule?

Istio separates traffic routing logic from service deployment.

Key resources:

1. VirtualService – Defines how requests are routed.
2. DestinationRule – Defines policies applied after routing (subsets, load balancing, TLS settings).

Traffic flow: Client → VirtualService → DestinationRule → Pods Canary Deployment

Example: Assume two versions: v1 (stable) and v2 (canary).

Step 1: DestinationRule with subsets

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
     name: my-service
spec:
host: my-service
subsets:
- name: v1
  labels:
version: v1
- name: v2
  labels:
    version: v2
```

Step 2: VirtualService with weighted routing

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
name: my-service
spec:
   hosts:
       - my-service
http:
- route:
- destination:
host: my-service
subset: v1
weight: 90
destination:
host: my-service
subset: v2
weight: 10
```

This sends 90% traffic to v1 and 10% to v2. Gradually increase v2 weight to shift traffic. Rollback is immediate by adjusting weights.

4. What is the Istio Ingress Gateway, and how does it differ from a standard Kubernetes Ingress controller?

The Istio Ingress Gateway is an Envoy-based proxy that manages inbound traffic from outside the cluster into the service mesh.

- It is configured using: Gateway resource (exposed ports and TLS settings), VirtualService (routing rules). It integrates directly with Istio's traffic management, security, and observability.
- Standard Kubernetes Ingress Controller: Exposes HTTP/HTTPS routes. Works independently of the service mesh. Typically supports basic Layer 7 routing.

Key Differences:

- Kubernetes Ingress: Simpler setup, Limited traffic control, No native mTLS inside the cluster
- Istio Ingress Gateway: Advanced routing (weights, header-based routing, fault injection), Full integration with mesh security, Consistent policy enforcement, Enhanced observability. The Istio gateway is mesh-aware; a standard Ingress controller is not.

5. How would you use Istio to improve observability, and how does it integrate with Prometheus, Grafana, and Jaeger?

Istio automatically collects telemetry from Envoy sidecars, providing visibility into service communication without major code changes.

- Metrics – Prometheus: Envoy proxies expose metrics. Prometheus scrapes these metrics. Metrics include latency, request count, and error rate.

- Visualization – Grafana: Grafana dashboards visualize Prometheus metrics. shows traffic distribution, success rate, and response times.
- Distributed Tracing – Jaeger: Istio integrates with Jaeger. Each request receives a trace ID. You can track request flow across microservices.

Benefits:

- ✓ Consistent telemetry across all services
- ✓ Service dependency graphs
- ✓ Golden signals (latency, traffic, errors, saturation)
- ✓ Faster root cause analysis Observability is handled at the proxy layer, ensuring standardized monitoring across the entire mesh.